SCORE: A Framework for Synthesizing Complete, Owned, Responsive Environments

J.P. Kardolus, Babon Innovations B.V., Utrecht, The Netherlands

Working draft, 12 October 2026

Abstract

Games, films and robot training need 3D worlds that are complete, editable and owned by the people who make them. World models and one-shot 3D generators can produce a convincing place from a sentence or a picture, but they give frames that no engine can open, or one fused scene without separate objects or clear rights. Studios have always built worlds in stages, from concept art and layout to modelling, surfacing, sound and review. We present SCORE, a world generation framework that keeps these stages and automates large parts of them with open vision foundation models, procedural tools and coding agents, while the creator chooses and reviews. Because every stage's output is kept, the creator can rerun or fix any single stage, and because every model allows commercial use, the creator stays the author of the result, holding its full commercial rights. We are building SCORE's first world, the game world of 2099, only 3 of whose 18 places passed its completion gate when it was last measured, and plan three further worlds of different kinds. Our aim is to show that checkable stages, with open models for shape and closed, paid models for pictures and surfaces, can build worlds that a creator owns and keeps editing. Code and paper are available at https://github.com/Babon-Innovations-b-v/score (MIT license).

Gap: teaser figure. This figure will show four worlds made with SCORE: the game world of 2099, a stylised underwater world, a simulation world for robots, and a third-person fantasy world. Each will appear as its chosen concept beside the finished world in its engine. It waits until the four worlds are finished and accepted by their creators.

1 Introduction

Interactive 3D worlds are the raw material of games, film and embodied AI [1], and building one is expensive: every object has to be modelled, surfaced, placed, given collision, lit and given sound in one style, and the world has to stay editable. Generative models promise to take over much of this, but in GDC's 2026 survey 52 percent of game workers thought generative AI was hurting the industry [2]; a tool for building worlds has to leave the result in the creator's hands.

World models generate a world as frames [3], [4] that no engine can open. 3D world generators [5], [6] produce geometry, but mostly as one fused asset, from which separate objects are recovered after the fact [7], [8]. Both work in one shot, so a creator cannot find or change the step behind a fault. Procedural systems [9], [10] and coding agents that write scenes as programs [11], [12], [13], [14] keep the world explicit, but rebuild a scene that already exists or draw their objects from a fixed library, and such agents judge their own geometry unreliably [15]. Open vision foundation models [16] now read shapes [17], masks [18], depth [19] and parts [20] out of pictures, and a coding agent

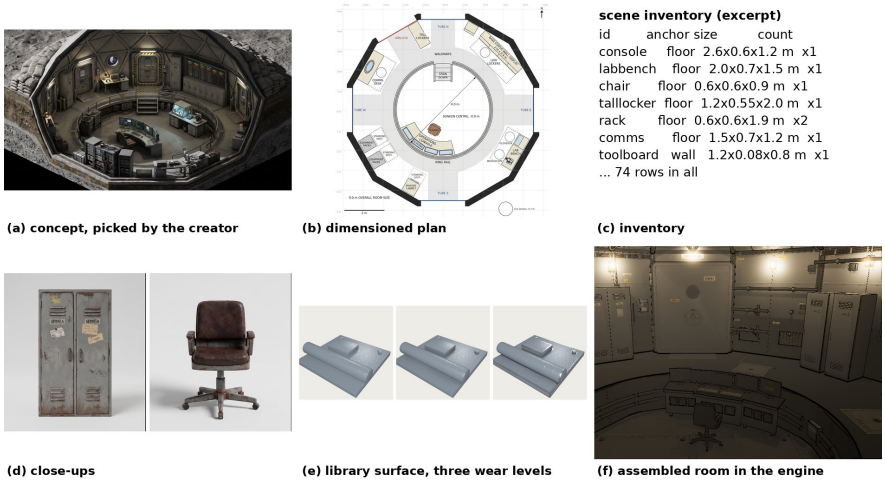


Figure 1: The stages on the hub of the game 2099: (a) the concept the creator chose, (b) the plan in metres, (c) part of the inventory, (d) two close-ups, (e) one library surface at three wear settings, (f) the room in the game engine.

can connect them as tools; none of them provides is one style across objects that different models made, which the field names as an open problem [21], [22].

Studios build worlds in stages, each with an output a person can check. SCORE keeps these stages and automates large parts of them with the models above, procedural tools and coding agents, while the creator chooses and reviews. Its worlds are **complete**: every object, surface, light and sound exists wherever the player looks. They are **owned**: every model in the framework allows commercial use of its output, so the creator stays the author, holding the full commercial rights. They are **responsive**: every object has real collision, mass and material. And they can be any kind of **environment**. To our knowledge, no other openly available world generation work joins vision foundation models into engine-ready scenes that are built from reference pictures, steered by a person's choices at each step, and made only with models that allow commercial use of their output.

This paper makes three contributions:

- **The SCORE framework:** staged, retraceable generation from a creator's choices to one canonical scene: a generated base layer under the creator's edit layers (Section 2).
- **Mechanisms that keep worlds coherent and faults cheap:** checks before every paid stage, a parts check, a shared surface library that keeps shape and surface apart, and review tools (Section 3).
- **Evidence from four worlds of different kinds**, and a shared benchmark (Section 4).

2 The framework

Figure 1 shows the stages on one place, from the creator's concept to the assembled room. Every stage's output is kept, so the creator can trace a weak result to its stage and rerun or fix that stage alone. Appendix C lists every model, its licence, and what was automatic or done by hand.

Inventory. A coding agent cuts the concept into tiles and lists every visible element, and each element becomes a row, with its kind, anchor, size and count, or is dropped with a written reason.

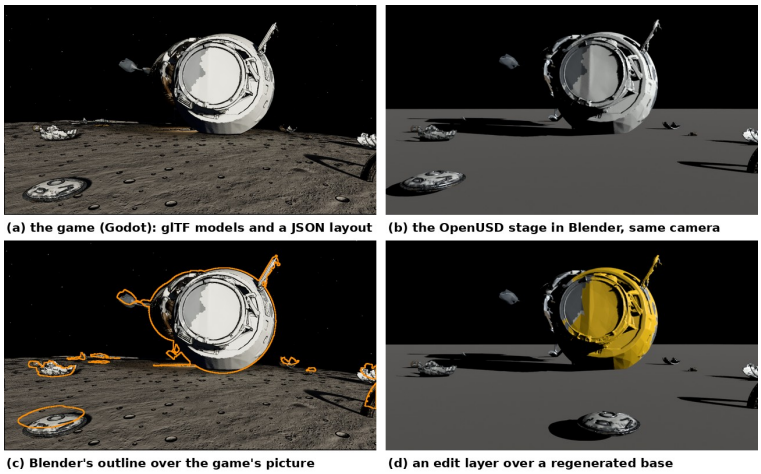


Figure 2: The wreck as an OpenUSD stage: (a) in the game; (b) rendered in Blender from the same camera; (c) Blender's object outlines over the game's picture; (d) after an edit in the edit layer (cover moved, hull given another surface) and a regenerated base.

Nothing is made or placed that is not a row.

Close-ups. Every inventory row gets a close-up, a picture of the object alone, drawn from its crop of the concept by an open model, Qwen-Image-Edit-2511 [23]; only a close-up that fails the shape check (Section 3.1) is redrawn by a closed one, Nano Banana Pro [24].

Models. Each object is built by a code builder that a coding agent writes or generated from its close-up by Pixal3D [17], as the parts check decides (Section 3.2). Every real-world object becomes one model: a console becomes a desk carrying separate monitors and keyboards. Models give shape only (Section 3.3). Outdoor ground follows a plan of flat areas and paths, with landforms from a picture drawn over it.

The canonical scene. Every place is one OpenUSD stage [25] in which each object carries its kind, its transform in metres, its collision and the library surface of each part, together with the room's shell, the ground, the backdrop, the lights and the cameras the place is judged from. The generated base layer lies under an edit layer that belongs to the creator, so an edit survives when the base is generated again (Figure 2).

The other layers. The same stage holds the rest of what makes a place complete, each in a layer of its own: people, as skinned characters that a chain of open models makes from a picture or a few words; sound, generated by MOSS-SoundEffect [26] and tied to surfaces, rooms and objects; motion, as time samples; particle effects; and shaders, baked into pictures any USD reader can draw. Every stage that loads a model runs as a batch on rented cards that are deleted afterwards, or as a job on a Kubernetes cluster.

Gap: the scene in a game engine. The same stage loaded in a game engine with its collision, surfaces, mass and friction. It waits on the game engine adapter.

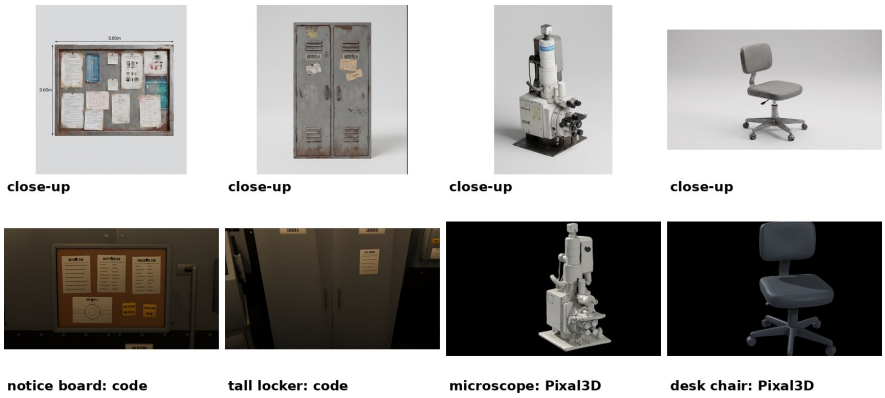


Figure 3: The parts check: four close-ups (top) and the objects as built (bottom). The notice board and locker are built in code, whose builds show every part; the microscope and chair are generated with Pixal3D.

3 Mechanisms

3.1 Checks before spending

Deterministic checks run before every stage that costs money, because a fault is free to fix in the plan and costly once built. They test real geometry: light leaks, blocked openings, doors, open, thin or leaning models, and colours off the palette. The close-ups' shape check measures framing and outline, and an open vision-language model, Qwen3.8-27B [27], compares each picture with its crop; it leans towards failing, because a wrong picture that passes costs a bad 3D model (Appendix C). Loose objects are dropped in a physics simulation, and one that would move further than a small limit fails the resting check, because a large move is a fault of the layout. A failed check sends the work back one stage (Appendix B).

Coding agents judging their own scenes agree with a measured score on geometry only 45.8 percent of the time [15], so a completion gate passes a place only when every inventory row has its object or a written reason, nothing visible stands in for a made piece, no resting fault is left, every model the creator picked is still the one shown, and every check ran on the stage as it is now; unknown blocks like fail, and a hook refuses to end an agent's session that claims a place is done while its gate does not pass.

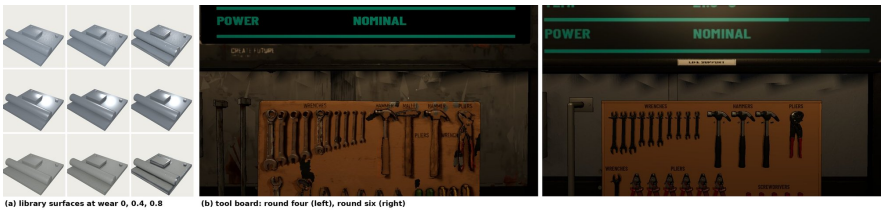
3.2 The parts check

Picture-to-3D models build chunky solids well, but flat panels, thin beams and openings badly, while code builders are exact but show only the parts the coding agent wrote. For each kind of object, the parts check renders a code build from the front: if every part of the close-up is visible in it, the kind is built in code, and otherwise it is generated and made solid (Figure 3).

Gap: the parts check's order. A queued change makes the parts check try generation first; this section is rewritten when it lands.

3.3 Shape and surface apart

Every part of every object takes one surface from a shared library of rule-based surfaces, written as ProcFunc functions [28] on Infinigen's shaders [9], coloured from the place's palette and worn only



(a) library surfaces at wear 0, 0.4, 0.8 (b) tool board: round four (left), round six (right)

Figure 4: (a) Three library surfaces at wear 0, 0.4 and 0.8. (b) The hub's tool board with each piece's own picture colours (left) and with library surfaces, one model per tool (right).

where wear has a cause, such as edges (Figure 4a, Appendix D). PartCrafter [20] splits a generated model into parts, and an open vision-language model, shown each part outlined on the close-up, names its library surface; the picture's colours are not kept. When each piece kept them, every piece brought its own rust and stains (Figure 4b); with shape and surface apart, the hub became consistent and the creator accepted it (Appendix A).

Gap: painting beyond the hub. On the rest of world 1 the painting is not yet consistent (Limitations). The surface stage's new method, to be measured there, is the finish-map route: Gemini 3.1 Pro [29] lists a model's finishes on its close-up and names a library surface for each, Nano Banana Pro draws the close-up again with one flat colour per finish, and SegviGen [30] lays that map onto the shape.

3.4 Review tools

A review page, built only from the files the stages wrote, shows each stage's output, before-and-after shots from the same cameras, every check's result, and the stage rendered by Blender along a walk through it. An optional world step can turn the concept into a walkable room to show what the concept does not; nothing it produces is shipped.

4 Evidence

We build four worlds with the same stages: the game world of 2099 (a Moon base, a camp on Mars and a prologue on Earth), a stylised underwater world, a simulation world for robots, and a third-person fantasy world.

Gap: Table 1, results per world and stage. Per world: places, time, cost, models built in code and generated, and faults caught before spending against those found after. It waits on the creator's acceptance of each place; worlds 2 to 4 are not started.

Gap: figure of world 1. Places of world 1 from its current run, rendered from their stages beside their concepts, and later one place of each further world. It waits on the creator's review of that run.

Gap: LEGO-Bench, ablations and style. Scores on LEGO-Bench [15]; runs with and without the world step and the checks; whether a place keeps one style. They wait on the benchmark's code and the surface stage's reviewed results.

With and without the agent tools. On two of three places of world 1, an agent with the completion gate and its tools used about half the output and cost of one without them; on the third, the one without left fewer faults. Each arm ran once (Appendix E).

5 Limitations

- **One creator, one world so far, and that world not yet accepted.** Every verdict is one creator's. When last measured, only 3 of world 1's 18 places passed their own completion gate, and the creator has accepted one place, the hub.
- **Code builders do not scale, and places come out sparser than their concepts.**
- **Two stages depend on closed, paid models.** Nano Banana Pro draws the concepts and every close-up that the open model's pictures fail; on the last 17 close-ups of world 1, none of the open model's pictures passed the shape check (Appendix C). The surface stage uses two of Google's models: Gemini 3.1 Pro lists each model's finishes on its close-up and names their library surfaces, and Nano Banana Pro draws the finish map that SegviGen lays onto the shape. A public framework thus depends on closed services for two of its stages. Their daily limits set the pace of world 1, and they can change their price or be retired, which limits reproducibility. An open-model fallback for both stages is future work.
- **Parts do not follow finishes, and shadow reads as paint.** A finish without its own part, such as a gold foil patch on a white hull, is lost when the part is painted with one surface, and the judge reads shadow on the close-up as dark paint.
- **Objects are physical but not interactive.**

6 Future work

Building the three further worlds; a game engine adapter that writes mass and friction; interactable objects, which robotics needs; a surface stage that holds on world 1; open close-ups that pass the shape check more often; and LEGO-Bench [15].

7 Conclusion

SCORE builds worlds from a creator's choices in checkable stages whose outputs are kept. On its first world, the parts check, one model per real-world object and the shared surface library made one room, the hub, consistent enough for its creator to accept it. Only 3 of that world's 18 places passed the completion gate when last measured; whether the framework holds across a whole world, and across kinds of worlds, is what the remaining places, worlds and the benchmark will show.

References

[1] Y. Yang *et al.*, "Holodeck: Language guided generation of 3D embodied AI environments," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[2] D. Argüello, "36 percent of game workers use generative AI, but half think it's bad for the industry." Game Developer, Feb. 2026.

[3] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, *et al.*, "Genie: Generative interactive environments," *arXiv preprint arXiv:2402.15391*, 2024.

[4] J. Parker-Holder and S. Fruchter, "Genie 3: A new frontier for world models." Google DeepMind blog, Aug. 2025. Available: https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/

[5] D. Wang, H. Jung, T. Monnier, K. Sohn, *et al.*, "WorldGen: From text to traversable and interactive 3D worlds," *arXiv preprint arXiv:2511.16825*, 2025.

[6] Tencent Hunyuan, "HY-World 2.0." GitHub repository, 2026. Available: https://github.com/Tencent-Hunyuan/HY-World-2.0

[7] J. Hu, J. Guo, J. Cen, C. Yang, S. Li, and W. Shen, "WorldAct: Activating monolithic 3D worlds into interactive-ready object-centric scenes," *arXiv preprint arXiv:2605.15843*, 2026.

[8] M. Niu, J. He, R. Yu, L. Fu, *et al.*, "WorldSculpt: Generating compositional worlds from grounded videos," *arXiv preprint arXiv:2609.05416*, 2026.

[9] A. Raistrick *et al.*, "Infinite photorealistic worlds using procedural generation," *arXiv preprint arXiv:2306.09310*, 2023.

[10] A. Raistrick *et al.*, "Infinigen Indoors: Photorealistic indoor scenes using procedural generation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[11] Z. Hu, A. Iscen, A. Jain, T. Kipf, *et al.*, "SceneCraft: An LLM agent for synthesizing 3D scene as Blender code," in *International conference on machine learning (ICML)*, 2024.

[12] Z. Li, Y. Liao, and B. Zhu, "Recursive code world models: Building complex worlds through recursive scene programs," *arXiv preprint arXiv:2609.11499*, 2026.

[13] C. Guo, J. Li, Y. Li, and Z. Huang, "WorldClaw: Agentic 3D open-world generation at scale," *arXiv preprint arXiv:2608.05248*, 2026.

[14] L. Yin, W. Cheng, Z. Qin, T. Huang, *et al.*, "AutoUE: Automated generation of 3D games in Unreal Engine via multi-agent systems," *arXiv preprint arXiv:2603.07106*, 2026.

[15] X. Li *et al.*, "LEGO-Anything: Coding agents for 3D scene reconstruction," *arXiv preprint arXiv:2609.36380*, 2026.

[16] M. Oquab, T. Darcet, T. Moutakanni, *et al.*, "DINOv2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[17] D.-Y. Li *et al.*, "Pixal3D: Pixel-aligned 3D generation from images," *arXiv preprint arXiv:2605.10922*, 2026.

[18] N. Carion *et al.*, "SAM 3: Segment Anything with concepts," *arXiv preprint arXiv:2511.16719*, 2025.

[19] R. Wang *et al.*, "MoGe-2: Accurate monocular geometry with metric scale and sharp details," *arXiv preprint arXiv:2507.02546*, 2025.

[20] Y. Lin *et al.*, "PartCrafter: Structured 3D mesh generation via compositional latent diffusion transformers," in *Advances in neural information processing systems (NeurIPS)*, 2025.

[21] J. Wu, Z. Lou, J. Liu, D. Du, C. Guo, and S. Guo, "From visual synthesis to interactive worlds: Toward production-ready 3D asset generation," *arXiv preprint arXiv:2604.23629*, 2026.

[22] Z. Yang *et al.*, "FlowScene: Style-consistent indoor scene generation with multimodal graph rectified flow," *arXiv preprint arXiv:2603.19598*, 2026.

[23] Qwen Team, "Qwen-Image-Edit-2511 model card." Hugging Face, 2025. Available: https://huggingface.co/Qwen/Qwen-Image-Edit-2511

[24] Google, "Gemini 3 Pro Image ('Nano Banana Pro') through the Gemini API." 2026. Available: https://ai.google.dev/

[25] Pixar Animation Studios and Alliance for OpenUSD, "OpenUSD: Universal scene description." 2026. Available: https://openusd.org/

[26] OpenMOSS-Team, "MOSS-SoundEffect v2.0." Hugging Face, 2026. Available: https://huggingface.co/OpenMOSS-Team/MOSS-SoundEffect-v2.0

[27] Qwen Team, "Qwen3.8-27B model card." Hugging Face, 2026. Available: https://huggingface.co/Qwen/Qwen3.8-27B

[28] A. Raistrick *et al.*, "ProcFunc: Function-oriented abstractions for procedural 3D generation in Python," *arXiv preprint arXiv:2604.26943*, 2026.

[29] Google, "Gemini 3.1 Pro (gemini-3.1-pro-preview) through the gemini API." 2026. Available: https://ai.google.dev/

[30] L. Li *et al.*, "SegviGen: Repurposing 3D generative model for part segmentation," *arXiv preprint arXiv:2603.16869*, 2026.

[31] Blender Foundation, "Blender." 2026. Available: https://www.blender.org/

[32] O. Siméoni, H. V. Vo, M. Seitzer, *et al.*, "DINOv3," *arXiv preprint arXiv:2508.10104*, 2025.

[33] K. Deng *et al.*, "GeoSAM2: Unleashing the power of SAM2 for 3D part segmentation," *arXiv preprint arXiv:2508.14036*, 2025.

[34] L. Yang, B. Kang, Z. Huang, *et al.*, "Depth Anything V2," *arXiv preprint arXiv:2406.09414*, 2024.

[35] World Labs, "Terms of Service." 2026. Available: https://docs.worldlabs.ai/terms-of-service

[36] Black Forest Labs, "FLUX.2 [klein] 4B model card." Hugging Face, 2026. Available: https://huggingface.co/black-forest-labs/FLUX.2-klein-4B

[37] D. Rempe, M. Petrovich, Y. Yuan, H. Zhang, X. B. Peng, *et al.*, "Kimodo: Scaling controllable human motion generation," *arXiv preprint arXiv:2603.15546*, 2026.

[38] P. BehnamGhader, V. Adlakha, M. Mosbach, D. Bahdanau, N. Chapados, and S. Reddy, "LLM2Vec: Large language models are secretly powerful text encoders," in *Conference on language modeling (COLM)*, 2024.

[39] A. Grattafiori, A. Dubey, A. Jauhri, *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[40] X. Yang, D. Kukreja, D. Pinkus, others, and K. Kitani, "SAM 3D Body: Robust full-body human mesh recovery," *arXiv preprint arXiv:2602.15989*, 2026.

[41] A. Qammaz, "SAM3DBody-cpp: A standalone C++ inference engine for SAM 3D Body." GitHub, 2026. Available: https://github.com/AmmarkoV/SAM3DBody-cpp

[42] M. Korosteleva and O. Sorkine-Hornung, "GarmentCode: Programming parametric sewing patterns," *ACM Transactions on Graphics*, vol. 42, no. 6, p. 197, 2023.

[43] The Newton Developers, "Newton: A GPU-accelerated physics simulation engine." 2026. Available: https://github.com/newton-physics/newton

[44] M. Macklin, "Warp: A high-performance Python framework for GPU simulation and graphics." NVIDIA, 2022. Available: https://github.com/NVIDIA/warp

[45] C. Ye *et al.*, "Hi3DGen: High-fidelity 3D geometry generation from images via normal bridging," in *Proceedings of the IEEE/CVF international conference on computer vision (ICCV)*, 2025.

[46] Black Forest Labs, "FLUX.2 [klein] Base 4B model card." Hugging Face, 2026. Available: https://huggingface.co/black-forest-labs/FLUX.2-klein-base-4B

[47] thedeoxen, "refcontrol: Reference-depth LoRA for FLUX.2 [klein] 4B." Hugging Face, 2026. Available: https://huggingface.co/thedeoxen/refcontrol-FLUX.2-klein-4B-reference-depth-lora

[48] J.-P. Zhang, C.-F. Pu, M.-H. Guo, Y.-P. Cao, and S.-M. Hu, "One model to rig them all: Diverse skeleton rigging with UniRig," *ACM Transactions on Graphics (SIGGRAPH)*, 2025.

[49] R. Abdrashitov, K. Raichstat, J. Monsen, and D. Hill, "Robust skin weights transfer via weight inpainting," in *SIGGRAPH asia 2023 technical communications*, 2023.

[50] Anthropic, "Claude Code." 2026. Available: https://www.anthropic.com/claude-code

[51] H. I. I. Tam, H. I. D. Pun, A. T. Wang, A. X. Chang, and M. Savva, "SceneEval: Evaluating semantic coherence in text-conditioned 3D indoor scene synthesis," *arXiv preprint arXiv:2503.14756*, 2025.

[52] H. Xia *et al.*, "SAGE: Scalable agentic 3D scene generation for embodied AI," *arXiv preprint arXiv:2602.10116*, 2026.

Appendix

A The hub in six rounds

The mechanisms of Section 3 came out of six rounds of work on the hub on 6 and 7 October 2026, each answering the creator's verdict on the round before (Table A1). In the first two rounds, he chose between the old and the new method side by side from the same cameras.

Table A1. The hub's six rounds. Cost is given as rented GPU time in euros and picture-model calls in dollars. Frame time is the mean frame time on an RTX 5080, in the night scene, at 1600×900.

Round	Change	Creator's verdict	Cost	Frame time
1, two walls, A/B	surface library, routing by shape, room checks	chose the new method ("Y looks much cleaner")	€0.88	7.81 ms vs 7.53
2, two walls, A/B	71-variant library, shared texture sets, roof check	chose the new method again	€1.70	8.05 ms vs 8.24
3, whole room	all stages end to end, 74 models	old models were still loaded; labels and floor fittings were wrong	€1.67	6.78 ms (old 9.22)
4, whole room	detail through the pipeline, picture colours kept	"pretty much all of this became messy"	€8.50, $5.09	9.6 ms
5, eight pieces, A/B	shape only from the pipeline, library surfaces, parts check	new method preferred for the door, notice board and porthole; three pieces still wrong	€0.21	not run
6, whole room	round 5 everywhere, one model per object, straightness check	"this looks amazing, I have no negative feedback"	€3.48, $1.07	5.6 ms

Three findings shaped the method. First, deterministic checks catch what agents miss: in round one, the share of rays leaking out through walls and roof fell from 1.53 percent of 5,817 to zero, and

the colour noise within surfaces from a median of 31.1 to 3.2 percent, but only walking through the loaded room showed 26 old models still in it, which became the made-only check. Second, generated pieces that kept their pictures' colours made the room inconsistent, so surfaces come only from the library. Third, generated composite objects melted their parts together, and generated boxes leaned (one locker by 13 degrees), which led to one model per real-world object and to the straightness check (Figure 4b). By round six the room used 0.45 million triangles, against 1.96 million in round four, and 187 MB on disk, against 487 MB.

B Check details

- **Room checks** cast rays to find light leaking out through walls and roof, find pieces in front of a real opening and roof and wall fittings more than 3 cm off their surface or turned more than 5 degrees from it, and count pieces, triangles and lamps against the room's budget.
- **Door check:** every door must stand in a wall or partition that fully separates its two sides, tested by walking a grid from one side to the other.
- **Parts check:** in the code build, every part shown in the close-up must be at least 10 percent visible from the front, and no label may cover a part.
- **Model check:** every model must be a closed solid with walls at least 3 mm thick. A generated box fails if one of its sides tilts more than 2 degrees or is warped. Making a generated model solid stops if its bounding box grows more than 3 percent.
- **Palette sweep:** every model's colours are compared with the place's library palette.
- **Made-only check:** in the engine, any visible mesh that the framework did not make fails. A made-only finding can never be waived.
- **Close-up shape check:** the picture needs a plain border all round, one object filling a sensible share of the frame, and an outline within a set proportion of the inventory row's box. The judge's seven questions (among them one object, a clean background, the whole object, the same object, no parts missing or added, the right proportions) each pass by a majority of three answers sampled with different seeds. A close-up passes only when both halves pass.
- **Settling and resting check:** an object that would turn more than 15 degrees or drift more than 30 cm when dropped keeps its laid pose and is marked as not resting; small debris, with no side over 1.4 m, may turn freely and drift up to 1 m. Hung objects are exempt by their inventory anchor, and fixed objects are not judged for tipping.
- **Completion gate:** the requirements given in Section 3.1, each with its result tied to a fingerprint of the stage's files and inputs; a result made on another stage counts as unknown. The placeholder check judges every mesh a code builder draws, and only the shell builders pass unnamed.

C Models and licences

Every model in the framework must allow commercial use of its output (Table C1); we read each licence for its clauses on commercial use. Candidates were dropped under this rule, among them NVIDIA's Lyra 2.0 (weights for internal research only), Hunyuan3D 2.1 (licence does not apply in the European Union) and GarmentCode's fork of NVIDIA Warp (non-commercial research only), which Newton's cloth solver on upstream Warp replaced; Blender's cloth simulation [31] is kept as a second route. The one open doubt is the face LoRA, whose training data is not disclosed.

Table C1. Models in the framework.

Model	Role	Licence
Nano Banana Pro [24]	concepts, close-ups the open model fails, ground skins	paid API
Nano Banana Pro [24]	finish maps: the close-up drawn again with one flat colour per finish	paid API
Gemini 3.1 Pro [29]	each model's finishes listed on its close-up, and their library surfaces named	paid API
Qwen-Image-Edit-2511 [23]	close-ups, first	Apache-2.0
Qwen3.8-27B [27]	the close-ups' shape check; each part's material	Apache-2.0
Pixal3D [17]	picture to 3D	MIT; DINOv3 licence [32] for its encoder
PartCrafter [20]	parts	MIT; its non-commercial background remover not used
SegviGen [30]	lays the close-up's finish map onto the 3D shape (surface stage)	MIT, on TRELLIS.2 (MIT); DINOv3 licence for its encoder; its non-commercial background remover and nvdiffrast not used
GeoSAM2 [33]	parts of our own model, seeded by the close-up's regions (trial)	Apache-2.0
Depth Anything V2 Small [34]	ground relief	Apache-2.0
ProcFunc, Infinigen shaders	surfaces	BSD-3-Clause
MOSS-SoundEffect v2.0, CLAP	sound	Apache-2.0
World Labs [35]	optional world step	paid; outputs owned by paid users
FLUX.2 klein 4B [36]	earlier prop pictures; the characters' A-pose pictures from words and their hair's clay pictures	Apache-2.0
MoGe-2 [19], SAM 3 [18]	measuring and finding objects in pictures	MIT; SAM Licence
Kimodo [37]	the characters' motion from sentences	Apache-2.0 code; NVIDIA Open Model License for the Kimodo-SOMA-RP-v1.1 weights, commercial use allowed

Model	Role	Licence
LLM2Vec [38] on Meta-Llama-3-8B-Instruct [39]	Kimodo's text encoder, used only while clips are made; nothing from it ships	Llama 3 Community License: commercial use allowed, the 700 million monthly users clause does not apply, "Built with Meta Llama 3" shown, no exclusion in the European Union for the text-only 8B model
SAM 3D Body [40], SAM3DBody-cpp [41]	body shape from a picture	SAM Licence for the weights, SAM3DBody-cpp's converted ONNX and GGUF weights included whatever their tag says; MIT for SAM3DBody-cpp's code
SOMA-X and MHR	the characters' bodies	Apache-2.0
GarmentCode [42], Newton [43] on Warp [44], Blender's cloth [31]	the clothes' sewing patterns and their drape	MIT; Newton and Warp Apache-2.0; Blender is GPL and its output is ours; GarmentCode's Warp fork (non-commercial research only) not used
Hi3DGen [45]	the characters' hair	MIT code; MIT and Apache-2.0 weights
FLUX.2 klein base 4B [46] with the refcontrol reference-depth LoRA [47]	the characters' faces	Apache-2.0, both; the LoRA's training data is not disclosed
UniRig [48]	the skeleton and skin of a quadruped	MIT code and weights
Robust Skin Weights Transfer [49]	the SOMA-X skeleton's skin weights moved onto a Pixal3D person (a trial)	MIT reference code, ported with trimesh and SciPy; libigl, whose bindings carry GPL-3.0, not used
MakeHuman eyes	the characters' eyes	CC0

The close-ups' shape check was tuned on 40 close-ups that four open picture models drew of the lab's ten objects, each scored by hand, and on Nano Banana Pro's ten pictures of the same objects. With the majority of three sampled answers, the check agreed with the hand score on 29 of the 40, passed all ten of the closed model's pictures and two marginal ones, passed none scored wrong or failed, and failed nine that were good or usable. The judge's single answer at temperature zero was rejected: asked the same 51 questions twice in one batch, it changed its verdict on 7. On the lab, 7 of the closed model's first pictures failed for drawing the room or dimension lines around the object, and all 7 passed when redrawn from the crop alone. Table C2 gives the stage's runs on world 1.

Table C2. The close-up stage on world 1. Cloud is rented card time in euros; the closed model's pictures are in dollars.

Run	Close-ups	Open model accepted	Closed model accepted	Left for the creator	Cloud	Closed model
Lab, 22 rows (earlier check: one answer)	22	8	14	0	€4.12	$2.81
World 1's remaining rows (three places)	17	0	10	7	€3.06	$4.02

The picture-to-3D step, the part splitting, the surface bakes, the sound generation, all checks and the assembly run automatically. Coding agents (Claude Code [50]) write the plans, the inventories and every code builder; in the hub's final round, 86 of its 98 models were built in code. Some steps were still done by hand: some generated objects were turned to face the right way by eye, and a few labels were placed by hand before a rule took over that job. The concepts and the verdicts are the creator's own choices.

D The surface library

The surface library holds families of variants: painted, steel, aluminium, rubber, plastic, cable, fabric, composite, glass, screen, light and print. After the hub's second round it held 71 variants made from 14 ProcFunc recipes. Each variant is a recipe with settings, takes its colours only from the palette, and is baked into base-colour, metal-roughness and normal maps at a texture density that depends on how close the player can get. Wear has three levels and comes from causes: edges, and kicks within 32 cm of the floor that a piece stands on.

E With and without the agent tools

The tools that check an agent's work were tested on three places of world 1: a room (the workshop, 814 objects), an outdoor place (the street, 1,198 objects) and a dense room (the hangar, 1,657 objects). They are the completion gate (Section 3.1); a triage of the resting check, which judges each object by what holds it up, read from its inventory row, and keeps only real faults [51], [52]; a pick lock, which refuses an export that would show anything but the model the creator picked for a row; and a render-and-compare, which lists the objects least like their close-ups by DINOv2 similarity [16]. In both arms, the same agent (Claude Opus 5.5 in Claude Code, headless) started from the same exported stage and inventory and was asked to make the place as complete as it could in about 50 minutes, through its layout data only; one arm had the framework from before these tools. Each arm ran once, so Table E1 shows what happened, not an average.

Table E1. One session per arm and place. Faults are the resting triage's real faults left on the final stage, measured against the arm's own data; placeholders by the first placeholder check; rows are inventory rows with no object and no written reason. Agent cost is the session's model cost; cloud is its rented machines' cost.

Place, arm	Real faults (start → end)	Placeholders	Rows missing	Minutes	Agent cost	Output tokens	Cloud	Exports	Gate said unknown
Workshop, without	18 → 13	0 → 0	1 → 0	36	$2.26	30.4k	€0.85	7	not used
Workshop, with	18 → 0	0 → 0	1 → 0	26	$1.42	14.3k	€0.85	2	2 of 4 calls
Street, without	11 → 0	0 → 0	0 → 0	38	$2.40	30.0k	€0.32	9	not used
Street, with	12 → 0	0 → 0	0 → 0	39	$1.21	10.4k	€0.76	1	0 of 5 calls
Hangar, without	14 → 4	5 → 0	0 → 0	47	$2.01	26.6k	€0.00	7	not used
Hangar, with	14 → 9	5 → 0	0 → 0	32	$2.34	29.1k	€0.64	5	3 of 6 calls

The tools made building cheaper more often than they made the scene better. On the workshop and the street, the arm with the tools used about half the agent's output and cost, and exported the stage once or twice instead of seven to nine times, because the triage named the few real faults among hundreds of alarms. On the workshop, 13 of its 18 faults were cleared by writing into the inventory what holds a piece up, which the creator still has to confirm. On the hangar the arm without the tools left fewer faults: the arm with them spent its time replacing five placeholder slabs and trying the drop test, which failed on a room with no ground mesh (fixed after the run), and the nine faults it left are models with stray parts below their feet. No agent claimed a place was complete, so the gate's hook never had to refuse a claim. No row carries an owner's pick, so the pick lock was not tested.